

Variabili

Le variabili sono "contenitori" che vengono utilizzati per memorizzare dei dati, come numeri o caratteri. Vengono memorizzati in memoria RAM e vi permangono solo per la durata del programma.

Tipi di dato

Una variabile è caratterizzata da un tipo, che va ad indicare per l'appunto il tipo di dato che esse contengono. In C esistono diversi tipi di dato:

- **int**: indica una variabile che memorizza un valore numerico intero, senza decimali, sia positivi che negativi (es: 9, -45), utilizzando 2 o 4 byte
- **float**: indica una variabile che memorizza un valore numerico in virgola mobile, ovvero con decimali, sia positivi che negativi (es: 9.99, -45.5), utilizzando 4 byte
- **double**: come il tipo float indica una variabile che memorizza un valore in virgola mobile, utilizzando però il doppio della memoria, ovvero 8 byte
- **char**: indica una variabile che memorizza un singolo carattere, racchiuso tra apici singoli (es: 'a', 'B')
- **bool**: indica una variabile che memorizza uno tra due possibili valori: true (VERO/ACCESO/SI) e false (FALSO/SPENTO/NO), ovvero un valore booleano. In C il tipo di dato bool non è tra quelli "di default" e deve, pertanto, essere incluso importando la libreria stdbool. N.B.: i valori booleani vengono interpretati come dei valori interi, dove 1 rappresenta true e 0 rappresenta false.

Casting

Per alcuni tipi di operazioni può essere necessario imporre che il valore finale assuma un ben specifico tipo. Per fare questo eseguiamo una operazione, detta casting, che serve proprio ad imporre un tipo di dato ad un valore, e si indica specificando tra () il tipo desiderato, prima dell'operazione.

Esempio:

```
int n1, n2, n3, somma;  
float media;  
  
somma = n1 + n2 + n3;  
media = (float) somma / 3;
```

In questo esempio calcoliamo la media di tre numeri interi; la media può assumere un valore con la virgola, pertanto è stata dichiarata come float.

Questo, tuttavia, non è sufficiente ad imporre che il risultato della divisione eseguita per ottenere la media sia un valore float, perché nel linguaggio C la divisione tra numeri interi restituisce un numero intero.

Dal momento che noi vogliamo mantenere le cifre decimali, effettuiamo il casting a float, ovvero imponiamo che il risultato della divisione tra interi non ci restituisca un valore intero, ma con la virgola.

Dichiarare una variabile

La dichiarazione è la fase in cui viene "creata" una variabile. Per farlo occorre indicare il tipo di dato che la variabile deve memorizzare e assegnare un nome (detto identificatore) con il quale si farà riferimento alla variabile all'interno del programma.

Quindi la sintassi è:

tipo nomeVariabile;

Possiamo dichiarare più variabili contemporaneamente (se sono dello stesso tipo) separandole con una virgola.

Esempio:

tipo variabile1, variabile2, variabile3;

È bene avere alcuni accorgimenti quando si assegna un nome ad una variabile:

- il nome deve essere univoco, ovvero non possiamo avere due variabili con lo stesso nome;
- i nomi possono contenere lettere, numeri e '_'
- i nomi possono iniziare esclusivamente con una lettera o un '_'
- i nomi non possono contenere spazi bianchi o caratteri speciali
- i nomi sono case-sensitive, ovvero sensibili a maiuscole e minuscole (es: myVar e myvar sono due variabili diverse)
- non è possibile usare parole riservate (es: int) come nomi di variabili.

Per il resto i nomi delle variabili sono liberi, ma è sempre meglio assegnare alle variabili dei nomi significativi, che rispecchino il loro contenuto. Una convenzione che si utilizza in linguaggio C è scrivere i nomi delle variabili tutti in minuscolo, mettendo le maiuscole solamente alle prime lettere di ogni parola nel caso di nome composto.

Assegnazione di una variabile

L'operazione di assegnazione consiste nell'attribuire un valore ad una variabile e viene effettuata utilizzando l'operatore '='.

La sintassi è la seguente:

nomeVariabile = valore;

N.B.: occorre tenere presente che quando assegniamo un valore ad una variabile andiamo a sostituire completamente il valore che aveva in precedenza, ovvero il valore nuovo viene sovrascritto a quello vecchio.

Quindi:

```
a = 3;  
a = 2;
```

dopo questa sequenza di istruzioni la variabile 'a' avrà valore 2.

È possibile effettuare l'assegnazione in fase di dichiarazione della variabile stessa; questa operazione, spesso, viene indicata come inizializzazione.

Esempi:

tipo *nomeVariabile = valore;*

tipo *variabile1 = valore, variabile2;*

Costanti

Come le variabili, anche le costanti le possiamo vedere come dei contenitori di dati. La sostanziale differenza sta nel fatto che, a differenza delle prime, le seconde mantengono il loro valore per tutta la durata del programma, e non è possibile modificarle.

Un modo per dichiarare una costante all'interno del nostro codice è quello di utilizzare la clausola:

```
#define NOMECONSTANTE valore
```

I nomi delle costanti seguono le stesse regole di quelli delle variabili. Per convenzione, i nomi delle costanti vengono scritti tutti in maiuscolo; non è richiesto a fini della corretta compilazione del codice, ma è una buona prassi da seguire per aumentarne la leggibilità.

Parlando di leggibilità del codice, un'altra buona prassi da seguire, che in C non è obbligatoria per la corretta compilazione, ma lo può essere in altri linguaggi come il Python, è l'utilizzo dell'indentazione. Questa prevede delle rientranze inserite attraverso una tabulazione (tasto TAB) ogni qualvolta delle istruzioni siano "all'interno" di un determinato blocco.

Facciamo un esempio:

```
#include <mialibreria.h>
int main(){
int a = 3, b = 2, somma;
somma = a + b;
return 0;
}
```

Il codice sopra riportato funziona correttamente, ma è "difficile" da leggere (essendo molto semplice è ugualmente di facile comprensione, ma per codici molto più lunghi si vede la differenza).

```
#include <mialibreria.h>

int main(){
    int a = 3, b = 2, somma;
    somma = a + b;

    return 0;
}
```

Il nuovo codice è identico al precedente, ma abbiamo evidenziato con una tabulazione tutte le istruzioni che sono contenute all'interno della funzione `main()`, che è tipicamente la funzione principale dei nostri programmi.

Inclusione di librerie

Spesso nei nostri programmi avremo bisogno di utilizzare delle funzioni che sono già pronte e messe a disposizione in librerie. Però per poterlo fare occorre importare tali librerie nel momento in cui dobbiamo utilizzarle; per fare ciò utilizziamo la clausola:

```
#include <nomelibreria.h>
```

Con questa "istruzione" stiamo chiedendo in fase di compilazione, di aggiungere al nostro codice la libreria indicata, perché ne utilizziamo una o più funzioni.